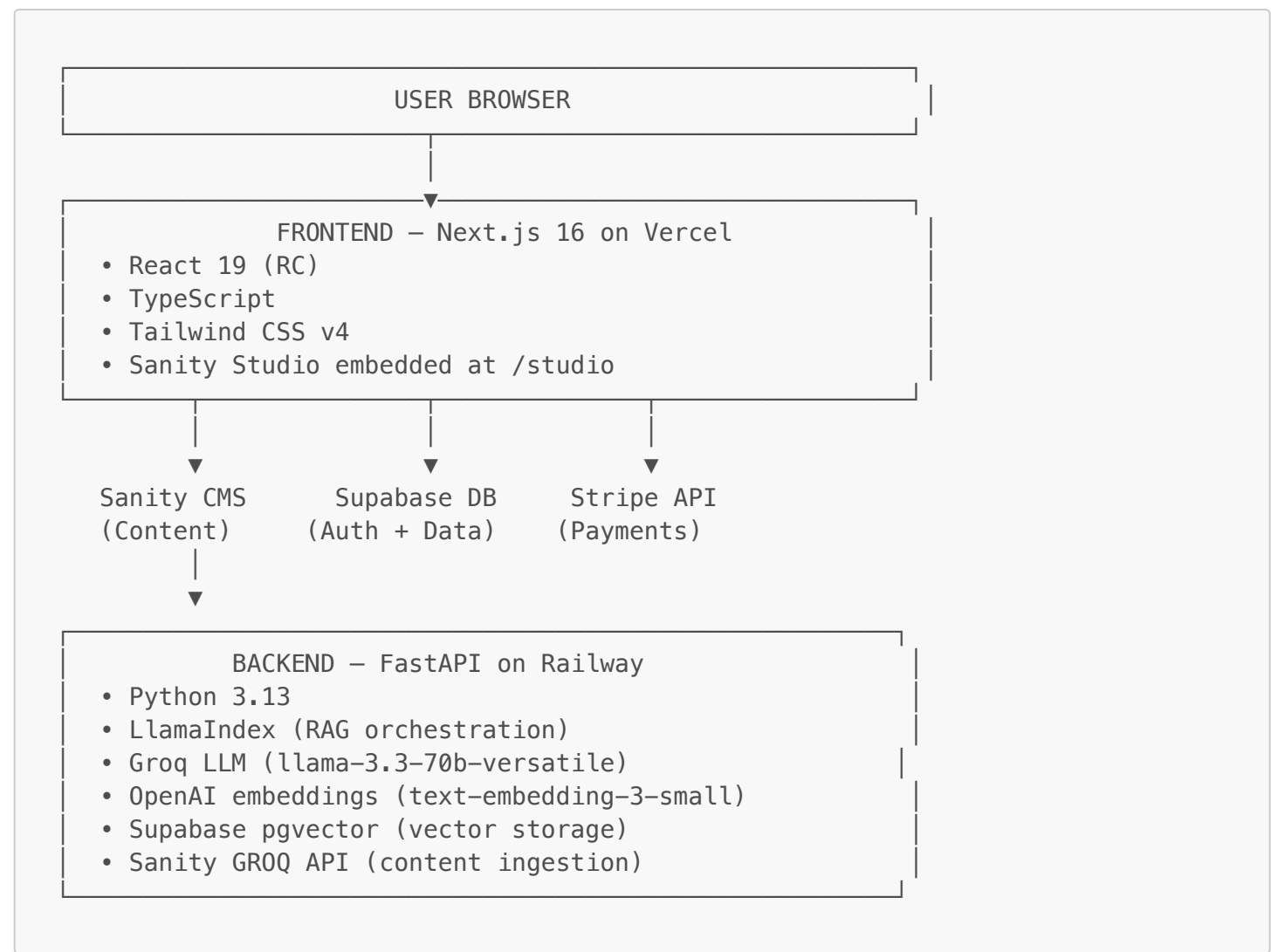


Health Transformation Review — Developer Guide

Architecture Overview



Repository Structure

Vermont-Health-Platform/	
├── backend/	Python FastAPI backend
│ ├── main.py	Single-file app (all logic here)
│ ├── requirements.txt	Pinned Python dependencies
│ ├── .env	Backend environment variables (git-ignored)
│ ├── .env.example	Template for .env
│ ├── railway.toml	Railway deployment config
│ ├── Procfile	Legacy process file
│ ├── data/	PDFs indexed into RAG
│ │ ├── Act 167 CE_St. Johnsbury Presentation_v04.pdf	
│ │ ├── Health Economics.pdf	
│ │ └── Wyman_Report.pdf	
│ ├── storage/	Local JSON vector store fallback
│ └── list_models.py	Utility: list available Groq models
└── frontend/	Next.js application
├── app/	Next.js App Router
│ ├── layout.tsx	Root layout (Header, AppShell, providers)
│ ├── page.tsx	Homepage
│ ├── globals.css	Global styles
│ └── api/	API Routes (server-side)
├── chat/	POST - proxy to Python backend
├── suggest/	POST - proxy to Python /api/suggest
├── search/	GET - Sanity full-text search
└── ticker/	GET - live news RSS feed

└─ subscribe/	POST – email list signup
└─ digest/	POST – weekly email via Resend
└─ health/	GET – backend health check
└─ rht-states/	GET – RHT state data
└─ state-metrics/	GET – state metric data
└─ stripe/	
└─ checkout/	POST – create Stripe session
└─ portal/	GET – customer billing portal
└─ webhook/	POST – handle Stripe events
└─ auth/callback/	Supabase OAuth callback
└─ studio/[...index]	Embedded Sanity Studio
└─ chat/	AI Analyst full page
└─ policy/	Policy hub + article pages
└─ economics/	Economics hub + article pages
└─ technology/	Technology hub + article pages
└─ clinical/	Clinical hub + article pages
└─ equity/	Equity hub + article pages
└─ academy/	Academy (courses, modules, etc.)
└─ dashboard/	State health dashboards
└─ states/	State profile pages
└─ account/	User account management
└─ pricing/	Pricing page + Stripe checkout trigger
└─ [many more pages]	
└─ components/	Shared React components
└─ AppShell.tsx	Layout shell (sidebars, ticker, breadcrumbs)
└─ Header.tsx	Sticky header with nav and ticker
└─ CollapsibleSidebar.tsx	Left/right sidebar wrapper
└─ RightSidebar.tsx	Inline AI chat widget
└─ HomeSidebar.tsx	Left navigation content
└─ CommandPalette.tsx	⌘K search overlay
└─ TickerStrip.tsx	Scrolling headlines component
└─ BackendStatus.tsx	Python backend health indicator
└─ NavDropdown.tsx	Header dropdown menus
└─ Footer.tsx	Site footer
└─ ConditionalFooter.tsx	Context-aware footer
└─ Breadcrumbs.tsx	Route breadcrumb trail
└─ ArticleEngine.tsx	Pillar article template
└─ templates/	Page layout templates
└─ ArticleEngine.tsx	
└─ AcademyModuleEngine.tsx	
└─ AcademyModuleLayout.tsx	
└─ HubPageTemplate.tsx	
└─ PillarHub.tsx	
└─ dashboard/	Dashboard-specific components
└─ NationalMap.tsx	
└─ RHTScorecard.tsx	
└─ USAMap.tsx	
└─ research/	Research tools
└─ APMCalculator.tsx	
└─ CEACalculator.tsx	
└─ HospitalFinancialScorecard.tsx	
└─ sidebar/	Sidebar sub-components
└─ AnalystNote.tsx	
└─ PolicyCalendar.tsx	
└─ SectorVitals.tsx	
└─ StateMonitor.tsx	
└─ academy/	Academy-specific components
└─ AcademyCard.tsx	
└─ KnowledgeCheck.tsx	
└─ LearningTracksHub.tsx	
└─ states/	State initiative map
└─ StateInitiativesMap.tsx	
└─ lib/	Shared utilities and clients
└─ auth.ts	Server-side auth helpers (getUser, requireAuth, requireRole)
└─ sanity.ts	Sanity client + imageUrlBuilder

└─ stripe.ts	Stripe client + PLANS config
└─ supabase.ts	Supabase anon client
└─ ticker.ts	Ticker data fetcher + defaults
└─ utils.ts	General utilities
└─ rate-limit.ts	Rate limiting helpers
└─ sanity-dashboard-queries.ts	Dashboard GROQ queries
└─ db/	Supabase data layer
└─ client.ts	db (anon) + dbAdmin (service role) clients
└─ academy.ts	Academy DB queries
└─ hospitals.ts	Hospital DB queries
└─ states.ts	State DB queries
└─ rht-profiles.ts	RHT profile queries
└─ learning-tracks.ts	Learning track queries
└─ state-initiatives.ts	
└─ time-series.ts	Time-series data queries
└─ data/	Static/seed data
└─ states.ts	State list
└─ hospital-data.ts	
└─ hti-timeseries-data.ts	
└─ learning-tracks-data.ts	
└─ performance-index-data.ts	
└─ rht-awards.ts	
└─ rht-program.ts	
└─ state-initiatives-data.ts	
└─ hooks/	
└─ useSolvencySimulation.ts	
└─ context/	
└─ DashboardContext.tsx	
└─ ProgramContext.tsx	
└─ sanity/	Sanity Studio and schemas
└─ sanity.config.ts	Studio configuration
└─ sanity.cli.ts	CLI config (project: fxz10xl7)
└─ structure.ts	Studio sidebar structure
└─ schemaTypes/	Document type definitions (21 types)
└─ lib/	Sanity client helpers
└─ scripts/	One-time data import scripts
└─ bulk_import.js	
└─ import-glossary.js	
└─ import_academy.js	
└─ seed-academy-content.js	
└─ seed-supabase.js	
└─ seed-ticker.js	
└─ reset-database.js	
└─ package.json	
└─ tailwind.config.js	
└─ tsconfig.json	
└─ next.config.js	
└─ .env.local	Frontend environment variables (git-ignored)

Local Development Setup

Prerequisites

- Node.js 20+
- Python 3.13+
- Git

1. Clone and install frontend

```
cd frontend
npm install
```

2. Configure frontend environment

Copy and fill out `.env.local`:

```
# Sanity (public – safe in browser)
NEXT_PUBLIC_SANITY_PROJECT_ID=fxz10xl7
NEXT_PUBLIC_SANITY_DATASET=production
NEXT_PUBLIC_SANITY_API_VERSION=2023-10-01
SANITY_API_TOKEN=sk... # Read/write token from Sanity
dashboard

# Supabase (public)
NEXT_PUBLIC_SUPABASE_URL=https://clryhwqaqhvdikgesjbc.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ... # Anon key from Supabase dashboard

# Supabase (server-only – never expose to browser)
SUPABASE_SERVICE_ROLE_KEY=eyJ... # Service role key from Supabase
dashboard

# Stripe (server-only)
STRIPE_SECRET_KEY=sk_live...
STRIPE_WEBHOOK_SECRET=whsec...
STRIPE_PRICE_SUBSCRIBER_MONTHLY=price...
STRIPE_PRICE_SUBSCRIBER_YEARLY=price...
STRIPE_PRICE_STUDENT_MONTHLY=price...
STRIPE_PRICE_STUDENT_YEARLY=price...
STRIPE_PRICE_PROFESSIONAL_MONTHLY=price...
STRIPE_PRICE_PROFESSIONAL_YEARLY=price...

# Email (server-only)
RESEND_API_KEY=re...
DIGEST_SECRET=some_random_secret_string

# Python backend URL (default: localhost:8000)
PYTHON_BACKEND_URL=http://localhost:8000

# App URL (for Stripe redirect URLs)
NEXT_PUBLIC_APP_URL=http://localhost:3000

# Optional: External ticker API
TICKER_API_URL=
```

3. Run the frontend

```
cd frontend
npm run dev
```

Frontend runs at `http://localhost:3000`. Sanity Studio is at `http://localhost:3000/studio`.

4. Configure backend environment

```
cd backend
cp .env.example .env
```

Edit `.env`:

```
# Required
OPENAI_API_KEY=sk-... # For text-embedding-3-small
```

```
GROQ_API_KEY=gsk_... # For llama-3.3-70b-versatile

# Sanity (for RAG content ingestion)
SANITY_PROJECT_ID=fxz10xl7
SANITY_DATASET=production
SANITY_API_TOKEN=sk...
SANITY_API_VERSION=2023-10-01

# Supabase (for pgvector and JWT auth)
SUPABASE_URL=https://clryhwqaqhvdikgesjbc.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...
SUPABASE_JWT_SECRET=your_jwt_secret # From Supabase: Settings > API > JWT Secret
SUPABASE_DB_URL=postgresql://postgres.[ref]:[password]@aws-0-us-east-1.pooler.supabase.com:6543/postgres

# CORS
FRONTEND_URL=http://localhost:3000

# Optional
INGEST_SECRET=some_random_secret # Protects /api/ingest endpoint
GROQ_MODEL=llama-3.3-70b-versatile # Override default model
```

5. Run the backend

```
cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Backend runs at <http://localhost:8000>. On first start it builds the RAG index (takes 1–3 minutes depending on content volume).

Frontend API Routes

All routes are in [frontend/app/api/](#).

POST /api/chat

Thin proxy to the Python backend. Validates request shape with Zod, then streams the response.

Request body:

```
{
  "message": "string (max 2000 chars)",
  "history": [{ "role": "user|ai", "text": "string (max 4000 chars)" }],
  "temperature": 0.7,
  "systemPrompt": "optional string (max 800 chars)"
}
```

Response: [text/plain](#) stream (token-by-token). Errors include [\[STREAM_ERROR\]](#) sentinel.

Auth: Forwards [Authorization](#) header from the incoming request to Python backend. The Python backend handles JWT validation. In dev mode (no [SUPABASE_JWT_SECRET](#) on backend), all requests are accepted.

POST /api/suggest

Proxy to Python [/api/suggest](#). Returns AI-generated follow-up questions.

Request: Same shape as `/api/chat`.

Response: `{ "suggestions": ["string", "string", "string"] }`

No auth required.

GET `/api/search?q=query`

Searches Sanity via GROQ across 6 content types.

Returns:

```
{
  "results": [{
    "_type":
    "post|policyAnalysis|academyModule|definition|caseStudy|analystNote",
    "_id": "string",
    "title": "string",
    "href": "string",
    "label": "Article|Policy Analysis|Academy Module|Definition|Case
    Study|Analyst Note",
    "description": "string|null"
  }],
  "query": "string"
}
```

POST `/api/stripe/checkout`

Creates a Stripe checkout session. Requires authenticated user.

Request: `{ "planId": "subscriber|student|professional", "interval": "monthly|yearly" }`

Response: `{ "url": "https://checkout.stripe.com/..." }`

Creates or reuses a Stripe customer record linked to the Supabase user.

POST `/api/stripe/webhook`

Handles Stripe events. Must be called by Stripe (signature verified with `STRIPE_WEBHOOK_SECRET`).

Events handled:

- `checkout.session.completed` → create subscription + grant role
- `customer.subscription.updated` → update subscription + re-grant role
- `customer.subscription.deleted` → downgrade to free
- `invoice.payment_failed` → set status to `past_due`

Idempotent — skips already-processed event IDs.

POST `/api/subscribe`

Adds email to Sanity `subscriber` collection. No auth required.

Request: `{ "email": "string" }`

Checks for existing subscriber; reactivates if previously deactivated.

POST `/api/digest`

Sends weekly HTML digest to all active subscribers with `digestEnabled: true`.

Requires `Authorization: Bearer $DIGEST_SECRET` header.

Fetches last 5 `policyAnalysis` documents from Sanity, builds HTML email, sends via Resend API.

Response: { "message": "Digest sent", "sent": N, "errors": N, "total": N }

GET /api/ticker

Fetches live headlines from RSS feeds (KFF Health News, Google News for FDA/CMS). Caches for 5 minutes.

Returns: { "headlines": [{ "text": "string", "url": "string" }] }

Python Backend API Routes

The backend runs at <http://localhost:8000> (dev) or Railway URL (prod).

POST /api/chat

RAG-enhanced streaming chat. Requires **Authorization: Bearer <supabase_jwt>** header. Subscriber role or higher required.

Request body:

```
{
  "message": "string",
  "history": [{ "role": "user|ai", "text": "string" }],
  "temperature": 0.7,
  "systemPrompt": "optional string"
}
```

Response: `text/plain` stream.

Auth flow:

- 1. Decode Supabase JWT using `SUPABASE_JWT_SECRET`
- 2. Extract `sub` (user_id) and `email` from JWT claims
- 3. Query `user_roles` table in Supabase
- 4. Determine highest role
- 5. Reject if role < subscriber

Dev mode: If `SUPABASE_JWT_SECRET` is not set, all requests are accepted as subscriber.

POST /api/suggest

Generate 3 follow-up questions. No auth required.

Uses last 6 messages of conversation history + current message. Calls Groq LLM to generate JSON array of 3 question strings.

Response: { "suggestions": ["q1", "q2", "q3"] }

POST /api/ingest

Triggers a background rebuild of the RAG index.

Requires **Authorization: Bearer \$INGEST_SECRET** header (if `INGEST_SECRET` is set).

Runs `build_index()` in a background asyncio task. Returns immediately.

Response: { "status": "accepted", "message": "Index rebuild started..." }

GET /health

Response:

```
{
  "status": "ok",
  "index_ready": true,
  "model": "llama-3.3-70b-versatile",
  "embedding_model": "text-embedding-3-small",
  "vector_store": "pgvector|local_json",
  "auth_enabled": true
}
```

Authentication System

Authentication uses Supabase Auth with SSR cookie-based sessions.

Role Hierarchy

Roles are ordered: `free` < `subscriber` < `student` < `professional` < `advisory` < `admin`

Server-side helpers (`lib/auth.ts`)

- `getUser()` — returns `AuthUser | null`. Fetches profile, roles, and subscription in parallel.
- `requireAuth()` — redirects to `/login` if not authenticated.
- `requireRole(role)` — redirects to `/upgrade` if role insufficient.
- `canAccess(role)` — returns boolean, no redirect.
- `roleAtLeast(userRole, required)` — compare roles without side effects.

Supabase Tables Used by Auth

- `profiles` — `id`, `full_name`, `avatar_url`
- `user_roles` — `user_id`, `role` (multiple rows per user; highest wins)
- `subscriptions` — `user_id`, `stripe_subscription_id`, `plan`, `status`, `current_period_start/end`, `cancel_at_period_end`
- `stripe_customers` — `user_id`, `stripe_customer_id`
- `stripe_events` — `event_id`, `event_type`, `payload` (idempotency log)

Auth Callback

`/auth/callback` — handles the OAuth/magic link redirect from Supabase.

Supabase Database Clients

Two clients in `lib/db/client.ts`:

```
// Anon client — respects Row Level Security
// Safe in React Server Components and client components
export const db = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);

// Service role client — bypasses RLS
// Use ONLY in server-side code (API routes, server actions)
// Never import in client components
export const dbAdmin = createClient(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY);
```

Sanity CMS

Project

- **Project ID:** `fxz10xl7`
- **Dataset:** `production`

- **API Version:** 2023-10-01
- **Studio URL:** https://your-frontend.vercel.app/studio

Client (lib/sanity.ts)

```
import { client, urlFor } from "@lib/sanity";

// Fetch content
const data = await client.fetch(groqQuery, params);

// Generate image URLs
const imageUrl = urlFor(imageRef).width(800).url();
```

Sanity document types

See CONTENT_GUIDE.md for full schema documentation.

Stripe Integration

Plans defined in lib/stripe.ts

Plan ID	Role Granted	Monthly	Yearly
subscriber	subscriber	\$29	\$279/yr
student	student	\$49	\$479/yr
professional	professional	\$99	\$959/yr

Advisory tier is handled manually — contact form at /advisory/contact.

Price IDs

Stored as environment variables:

```
STRIPE_PRICE_SUBSCRIBER_MONTHLY
STRIPE_PRICE_SUBSCRIBER_YEARLY
STRIPE_PRICE_STUDENT_MONTHLY
STRIPE_PRICE_STUDENT_YEARLY
STRIPE_PRICE_PROFESSIONAL_MONTHLY
STRIPE_PRICE_PROFESSIONAL_YEARLY
```

Webhook Flow

1. User completes Stripe checkout
2. Stripe sends checkout.session.completed to POST /api/stripe/webhook
3. Webhook retrieves subscription from Stripe
4. Upserts record in subscriptions table
5. Removes free role, adds the plan's role to user_roles

Key Frontend Dependencies

Package	Version	Purpose
next	^16.1.6	Framework
react	19.0.0-rc	UI library
@supabase/ssr	^0.9.0	SSR-safe Supabase auth

Package	Version	Purpose
@supabase/supabase-js	^2.89.0	Supabase client
sanity	latest	Sanity Studio
next-sanity	latest	Sanity client for Next.js
stripe	^20.4.1	Stripe server SDK
@stripe/stripe-js	^8.11.0	Stripe browser SDK
react-markdown	^10.1.0	Render AI responses as Markdown
@heroicons/react	^2.2.0	Icon library
d3-geo	^3.1.1	Geographic projections for maps
react-simple-maps	^1.0.0	Map components
tailwindcss	^4.0.0-beta.9	Styling
uuid	^13.0.0	UUID generation for message IDs

Key Backend Dependencies

Package	Version	Purpose
fastapi	~0.127.0	Web framework
uvicorn[standard]	~0.40.0	ASGI server
llama-index-core	~0.14.10	RAG orchestration
llama-index-llms-groq	~0.3.0	Groq LLM integration
llama-index-embeddings-openai	~0.3.0	OpenAI embeddings
llama-index-vector-stores-postgres	~0.4.0	pgvector store
llama-index-readers-file	~0.5.5	PDF reader
pypdf	~6.4.0	PDF text extraction
supabase	~2.15.0	Supabase Python client
PyJWT	~2.10.0	JWT verification
httpx	~0.28.0	Async HTTP client (Sanity API)
pydantic	~2.12.0	Request validation
asyncpg	~0.30.0	Async PostgreSQL driver

Deployment

Frontend — Vercel

- **Project ID:** prj_LVh0D2YKzL0vQQBGNYpkZSgulcpG
- Deploys automatically from `main` branch pushes
- All `frontend/` environment variables must be set in the Vercel project settings
- `PYTHON_BACKEND_URL` must point to the Railway backend URL in production

```
# Deploy manually
cd frontend
npx vercel --prod
```

Backend — Railway

Configuration in `backend/railway.toml`:

```
[build]
builder = "nixpacks"

[build.nixpacksConfig]
providers = ["python"]

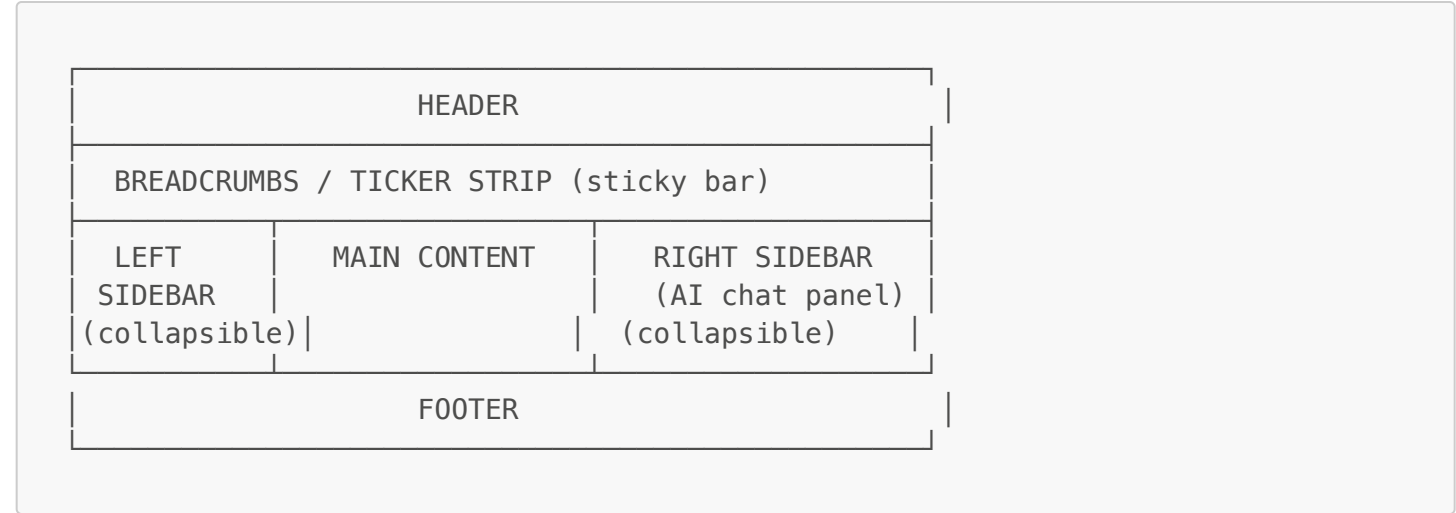
[deploy]
startCommand = "uvicorn main:app --host 0.0.0.0 --port $PORT"
restartPolicyType = "on_failure"
restartPolicyMaxRetries = 3

[healthcheck]
path = "/health"
intervalSeconds = 30
timeoutSeconds = 10
```

Railway provides `$PORT` automatically. All environment variables from `backend/.env` must be set in Railway project environment settings.

Layout System

The app shell (`components/AppShell.tsx`) manages the three-panel layout:



Sidebar behavior by route:

Route Pattern	Left Sidebar	Right Sidebar
<code>/</code> (homepage)	Open	Open
<code>/articles/*</code> , <code>/policy/*</code> , <code>/economics/*</code> , <code>/technology/*</code>	Closed	Closed
<code>/academy/modules/*</code> , <code>/academy/courses/*</code>	Closed	Closed
<code>/chat</code>	Hidden entirely	Hidden entirely
<code>/studio</code>	Hidden entirely	Hidden entirely
Other routes	Default	Default

On screens < 1024px wide the left sidebar collapses. On screens < 1280px wide the right sidebar collapses.

SidebarContext shares state between Header (toggle buttons) and AppShell (sidebar rendering).

Extending the Application

Adding a new content pillar page

1. Create `frontend/app/[pillar]/page.tsx`
2. Add navigation items in `components/Header.tsx` in the appropriate `Items` array
3. Add Sanity content type or reuse `policyAnalysis` with new category values
4. Add the route to `lib/sanity.ts` search query if searchable

Adding a new API route

1. Create `frontend/app/api/[route]/route.ts`
2. Export named HTTP method handlers (`GET`, `POST`, etc.)
3. Use `dbAdmin` for server-side Supabase operations
4. Use `requireAuth()` or `requireRole()` from `lib/auth.ts` for protected routes

Adding a new Sanity document type

1. Create `frontend/sanity/schemaTypes/[typeName].ts`
2. Export it and add to `frontend/sanity/schemaTypes/index.ts`
3. Add to `frontend/sanity/structure.ts` for Studio sidebar organization
4. Add a GROQ query in `frontend/app/api/` or component if needed
5. If it should be indexed in RAG, add a query in `backend/main.py SANITY_QUERIES` dict and call `POST /api/ingest`

Triggering RAG re-indexing

```
curl -X POST https://your-backend.railway.app/api/ingest \
-H "Authorization: Bearer $INGEST_SECRET"
```

Or in dev (no `INGEST_SECRET` set):

```
curl -X POST http://localhost:8000/api/ingest
```

Environment Variables Reference

Frontend (`frontend/.env.local`)

Variable	Required	Description
<code>NEXT_PUBLIC_SANITY_PROJECT_ID</code>	Yes	Sanity project ID
<code>NEXT_PUBLIC_SANITY_DATASET</code>	Yes	Sanity dataset name
<code>NEXT_PUBLIC_SANITY_API_VERSION</code>	Yes	Sanity API version
<code>SANITY_API_TOKEN</code>	Yes	Sanity read/write token (server-only)
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Yes	Supabase project URL
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Yes	Supabase anon key
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Yes	Supabase service role key (server-only)
<code>STRIPE_SECRET_KEY</code>	Yes (prod)	Stripe secret key
<code>STRIPE_WEBHOOK_SECRET</code>	Yes (prod)	Stripe webhook signing secret
<code>STRIPE_PRICE_SUBSCRIBER_MONTHLY</code>	Yes (prod)	Stripe price ID
<code>STRIPE_PRICE_SUBSCRIBER_YEARLY</code>	Yes (prod)	Stripe price ID
<code>STRIPE_PRICE_STUDENT_MONTHLY</code>	Yes (prod)	Stripe price ID

Variable	Required	Description
STRIPE_PRICE_STUDENT_YEARLY	Yes (prod)	Stripe price ID
STRIPE_PRICE_PROFESSIONAL_MONTHLY	Yes (prod)	Stripe price ID
STRIPE_PRICE_PROFESSIONAL_YEARLY	Yes (prod)	Stripe price ID
RESEND_API_KEY	Yes (prod)	Resend email API key
DIGEST_SECRET	Yes (prod)	Secret to protect /api/digest
PYTHON_BACKEND_URL	Yes (prod)	Python backend URL
NEXT_PUBLIC_APP_URL	Yes (prod)	Frontend URL for Stripe redirects
TICKER_API_URL	No	External ticker data override

Backend (backend/.env)

Variable	Required	Description
OPENAI_API_KEY	Yes	OpenAI API key for embeddings
GROQ_API_KEY	Yes	Groq API key for LLM
GROQ_MODEL	No	Override default Groq model
SANITY_PROJECT_ID	Yes	Sanity project ID
SANITY_DATASET	No	Sanity dataset (default: production)
SANITY_API_TOKEN	Yes	Sanity read token
SANITY_API_VERSION	No	Sanity API version
SUPABASE_URL	Yes	Supabase project URL
SUPABASE_SERVICE_ROLE_KEY	Yes	Supabase service role key
SUPABASE_JWT_SECRET	Yes (prod)	JWT secret from Supabase dashboard
SUPABASE_DB_URL	Yes (prod)	Postgres connection string for pgvector
FRONTEND_URL	Yes (prod)	Frontend URL for CORS
INGEST_SECRET	No	Protects /api/ingest endpoint